

MECANISME PENTRU ADMINISTRAREA MEMORIILOR ÎNTR-UN SISTEM

Conținut laborator:

- 1. Obiectivul lucrării.**
- 2. Noțiuni teoretice:**
 - a. Segmentarea memoriei**
 - b. Paginarea memoriei**
 - c. Sistem de adresare segmentată a memoriei SRAM de pe machetele din laborator**
- 3. Cerințe laborator.**
- 4. Temă.**

Obiectivul lucrării:

În lucrul cu memorii de dimensiuni mari, una din problemele care apar sunt legate de modul în care sunt generate adresele care conduc la locațiile ce se doresc accesate. Rezolvarea acestei probleme este făcută prin intermedierea între procesor și memorie a unor module logice, a căror componentă de bază o constituie regiștrii, denumite unități de administrare a memoriei (Memory Management Unit – MMU). Această lucrare își propune studiul unităților de administrare ale memoriilor care există într-un sistem de calcul. Pe parcursul ei se vor construi module MMU, utilizând circuitele reconfigurabile FPGA, care vor asigura accesarea memoriilor din clasa SRAM existente pe machetele de laborator (cu organizarea 256ko) și un „procesor” realizat sub forma unui automat CROM în circuitul reconfigurabil (proiectat în laboratorul 4).

Laboratorul face o incursiune în mecanismele hardware care se află în spatele tehnicilor de accesare segmentată și paginată a memoriei, existente la microprocesoarele din calculatoare, dar prezintă și tehnici de proiectare cu circuite reconfigurabile pentru accesarea memoriilor RAM externe.

Noțiuni teoretice:

Unitatea de management a memoriei este un motor logic intermediar plasat între unitatea centrală UC (procesor) și memoria respectivului sistem. Scopul acestui bloc este acela de a translata adresele logice emise de către U.C în adrese fizice prin care se adresează efectiv memoria.

De obicei procesoarele nu au capacitatea directă de a adresa în întregime memoria de lucru deoarece numărul liniilor de adrese emise direct de către procesor este mai mic decât numărul liniilor de adrese necesare lucrului cu memoria principală a sistemului de calcul (adresă fizică).

Există două mecanisme de administrare a memoriei întâlnite în practică ce își propun să rezolve această problemă: administrarea segmentată și administrarea paginată.

a. Segmentarea memoriei

Se bazează pe existența unor regiștri specializați, denumiți regiștri segment. De exemplu, la familia de microprocesoare 8086 (din care se trag toate microprocesoarele existente în majoritatea calculatoarelor la ora actuală), există 4 regiștri

segment: CS (code segment), DS (data segment), ES (extra segment) și SS (stack segment).

Prin intermediul unui registru segment este controlată o zonă din memoria principală denumită segment de memorie. De exemplu, la 8086 cei 4 regiștri au fiecare 16 biți dimensiune și deci pot controla fiecare segmente de memorie de câte 64k fiecare, așa cum se poate observa în figura 1.

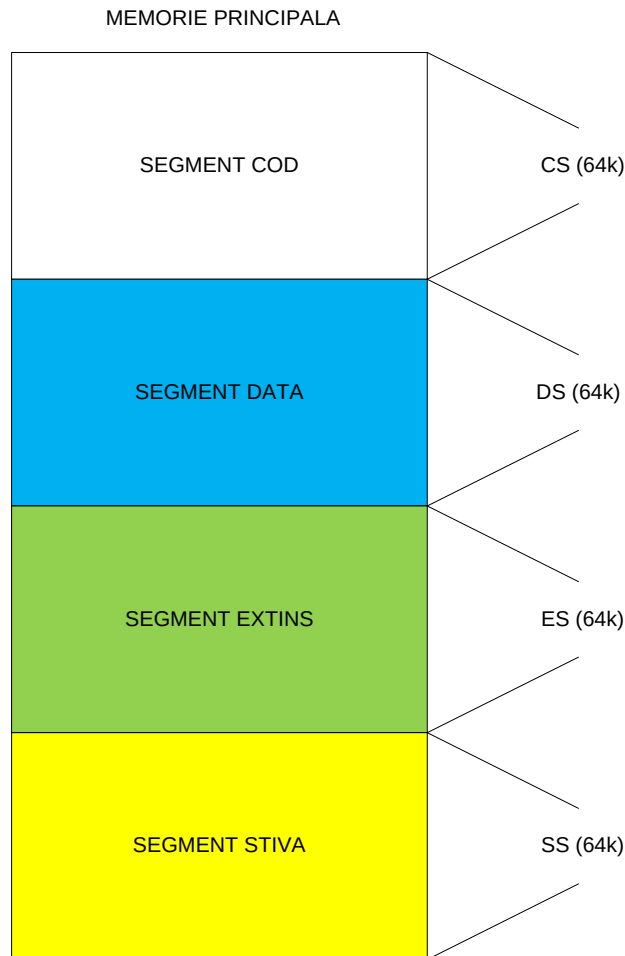


Fig.1. Memoria principală organizată segmentat (la 8086)

Zonele gestionate se pot suprapune (uneori total) sau pot fi independente, ocupând locații succesive de memorie. Prin program se poate modifica conținutul regiștrilor segment repositionând segmentele de memorie.

Denumirile segmentelor de memorie din figura 1 au legătură cu modul de operare al procesorului 8086. Astfel, segmentul cod conține instrucțiunile pe care procesorul le va prelua prin secvența de fetch pentru a le executa. Zonele segment data și segment extins sunt utilizate ca memorie de date unde sunt stocate variabile și șiruri de date. Zona segment stivă este utilizată pentru stiva. De aici se poate observa că administrarea segmentată a memoriei nu este importantă numai pentru accesarea de către procesor mai eficientă a memoriei (utilizând la un moment dat adrese logice de dimensiuni mai mici

decât adresele fizice) cât și pentru utilizarea aceleiași memorii în scopuri diferite (memorie de program, memorie de date, stivă).

Mecanismul de accesare al memoriei la administrarea segmentată a acesteia este prezentat în figura 2. Adresa logică emisă de procesor se adună cu adresa din registrul segment constituind o adresă fizică prin care se accesează memoria.

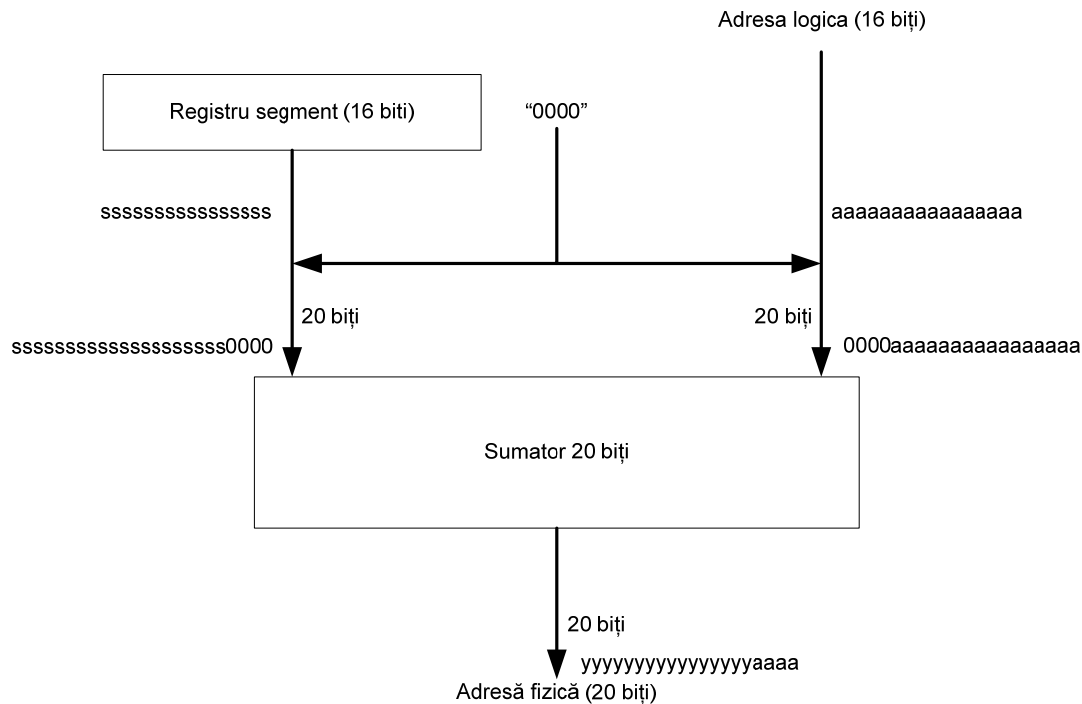


Fig.2. Mecanismul de adresare segmentată (8086)

Așa cum se poate observa în figura 2, registrul segment (care poate fi încărcat de asemenea de procesor) este întâi deplasat cu 4 poziții la stânga (echivalentul unei înmulțiri cu 16) și apoi este adunat cu adresa logică.

b. Paginarea memoriei

La administrarea paginată, memoria principală este organizată în pagini de dimensiuni egale, fiecare pagină având un număr de ordine specific și o adresă de început.

În procedura de traducerea a adresei logice emise de procesor către o adresă fizică de memorie intervine o structură hardware denumită tabelul paginilor (page table), tabel prin intermediul căruia se realizează asocierea între adresa logică și respectiv adresa fizică.

Tabelul paginilor este de fapt o porțiune de memorie RAM cu acces foarte rapid la care informația este divizată în mai multe câmpuri. Astfel, există câmpul care conține adresa fizică propriu-zisă precum și câmpuri ce indică controlul accesului în cadrul paginii respective de memorie.

Sistemul de adresare paginată este utilizat în special pentru administrarea memoriilor în sistemele cu memorie principală și secundară. Memoria secundară este constituită de

dispozitivele de stocare externe (hard disk, dvd rom, memorii cu acces USB) de capacitate mult mai mare decât cea a memoriei principale dar de viteză de acces redusă. Astfel, memoria totală de care beneficiază sistemul este privită ca fiind organizată pe pagini, ca în figura 3.

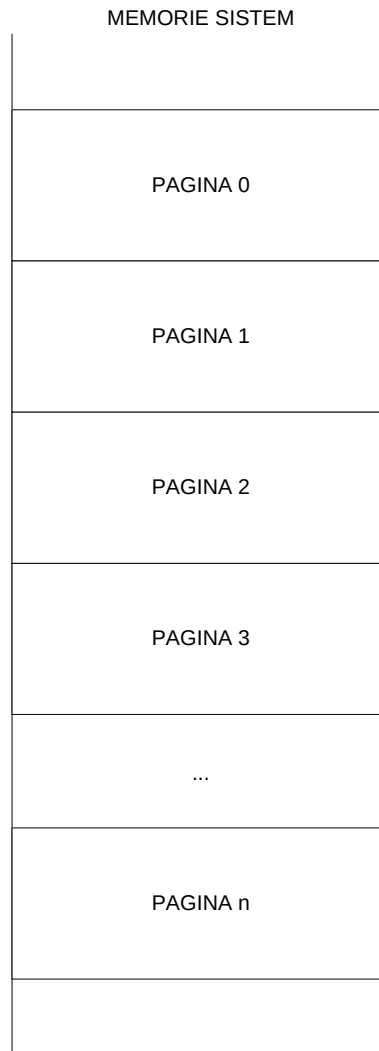


Fig.3. Organizarea memoriei unui sistem cu administrare paginată

Zona page table conține înregistrări cu informații despre fiecare pagina de memorie care există în sistem. O înregistrare conține adresa de început a paginii și o serie de biți de control al accesului la pagină (Control Access). Unul din aceștia informează dacă respectiva pagină se găsește sau nu în memoria principală a sistemului (bitul de prezență). Ceilalți biți de control dau informații despre accesele făcute anterior la pagina respectivă, pentru politica de înlocuire a paginilor din memoria principală (paginile neutilizate sau utilizate cu mult timp în urmă).

Mecanismul de adresare paginată este prezentat în figura 4.

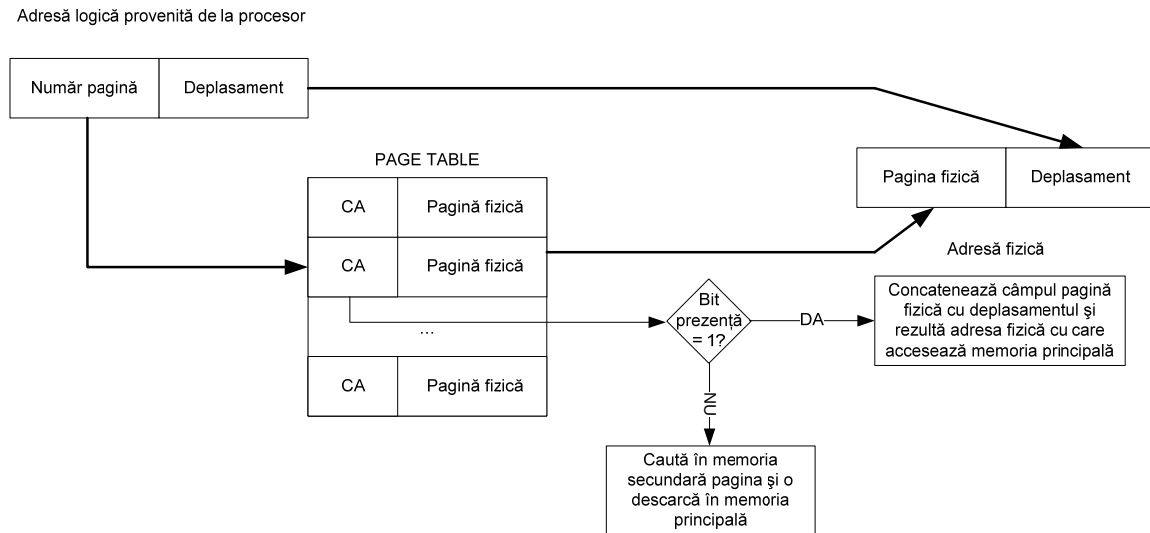


Fig. 4. Mecanismul de accesare a memoriei la adresarea paginată

Structura funcționează astfel:

- Adresa logică, provenită de la procesor, este compusă din două câmpuri: număr pagină și deplasament. Număr pagină reprezintă poziția unde se va găsi înregistrarea corespunzătoare din Page Table. Deplasamentul este utilizat în calculul adresei fizice.
- Utilizând câmpul Număr pagină se accesează înregistrarea corespunzătoare paginii din Page table. Aici se va găsi, printre altele, în câmpul CA, bitul care indică prezența, sau nu, a paginii solicitate în memoria principală. Dacă bitul este 1 înseamnă că pagina este prezentă în memoria principală, caz în care se concatenează adresa pagina fizică cu deplasamentul și se obține adresa fizică a locației care este solicitată. Dacă bitul prezență este 0 atunci pagina trebuie preluată din memoria secundară, scrisă în memoria principală și apoi actualizată Page table. În final se accesează conținutul paginii nou înscrise în memoria principală la deplasamentul conținut în adresa logică.

c. Sistem de adresare segmentată a memoriei SRAM de pe machetele din laborator

a. Memoria SRAM

Pentru ilustrarea modurilor de accesare a memoriei se vor utiliza modulele de memorie SRAM existente pe macheta de laborator cu Spartan 3.

Memoriile SRAM disponibile pe machetele de laborator sunt produse de firma Samsung și au codul de fabricație K6R4016V1D. Organizarea acestora este de 256kx16, și au următoarea structură la nivel de capsula:

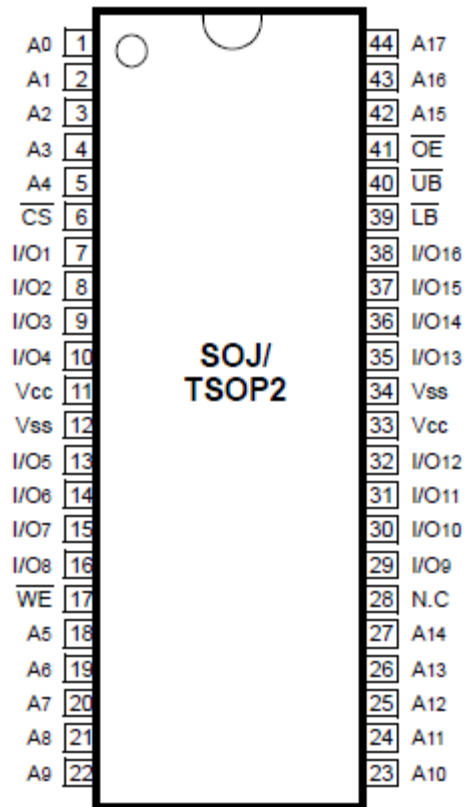
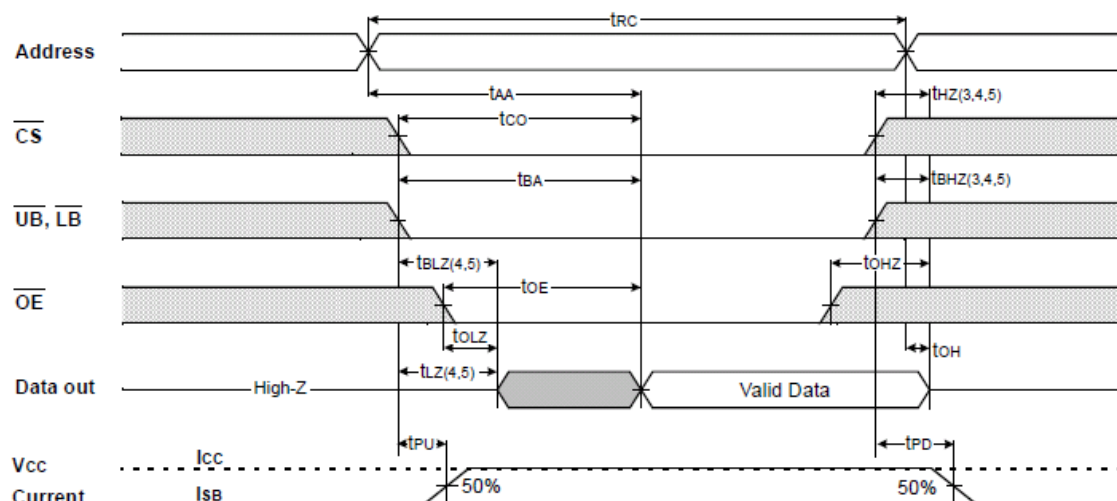


Fig.1. Organizarea capsulei pentru o memorie SRAM Samsung K6R4016V1D

Funcțiile pinilor circuitului sunt descrise în tabelul de mai jos:
Tabelul 1.

Nume pin	Funcție pin
A0 ... A17	Intrări de adrese
WE	Activare scriere (activ 0 logic)
CS	Selecție cip (activ 0 logic)
OE	Activare citire (activ 0 logic)
LB	Control ocmps date – I/O8 ... I/O1 (activ 0 logic)
UB	Control ocmps date – I/O16 ... I/O9 (activ 0 logic)
Vcc	Intrare alimentare (+3.3V)
Vss	Masă
N.C.	Neconectat

Formele de unda date în foaia de catalog pentru operația de citire sunt prezentate în figura de mai jos:



Parameter	Symbol	K6R4016V1D-08		K6R4016V1D-10		Unit
		Min	Max	Min	Max	
Read Cycle Time	t_{RC}	8	-	10	-	ns
Address Access Time	t_{AA}	-	8	-	10	ns
Chip Select to Output	t_{CO}	-	8	-	10	ns
Output Enable to Valid Output	t_{OE}	-	4	-	5	ns
$\overline{UB}$, $\overline{LB}$ Access Time	t_{BA}	-	4	-	5	ns
Chip Enable to Low-Z Output	t_{LZ}	3	-	3	-	ns
Output Enable to Low-Z Output	t_{OLZ}	0	-	0	-	ns
$\overline{UB}$, $\overline{LB}$ Enable to Low-Z Output	t_{BLZ}	0	-	0	-	ns
Chip Disable to High-Z Output	t_{HZ}	0	4	0	5	ns
Output Disable to High-Z Output	t_{OHZ}	0	4	0	5	ns
$\overline{UB}$, $\overline{LB}$ Disable to High-Z Output	t_{BHZ}	0	4	0	5	ns
Output Hold from Address Change	t_{OH}	3	-	3	-	ns
Chip Selection to Power Up Time	t_{PU}	0	-	0	-	ns
Chip Selection to Power Down Time	t_{PD}	-	8	-	10	ns

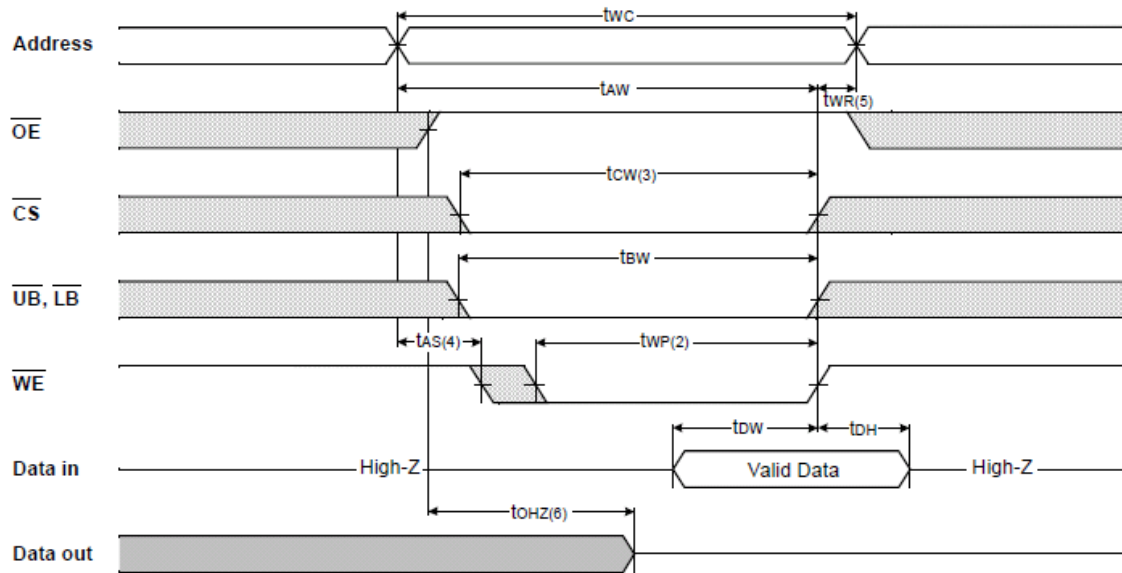
Fig.2. Secvența de citire pentru memoria SRAM Samsung K6R4016V1D (foaie de catalog)

Așa cum se poate observa în figura 2, pentru citire, după generarea adreselor locației, este plasat un semnal 0 logic pe intrarea CS pentru activarea circuitului urmat de plasarea unui semnal 0 logic pe intrarea OE pentru activarea liniilor de date ca ieșiri.

Cele două intrări $\overline{UB}$ și $\overline{LB}$ sunt utilizate pentru selectarea jumătății superioare respectiv inferioare din magistrala bidirecțională de adrese. Aceasta pentru a permite lucrul cu sisteme care pot prelua la un moment dat numai 8 biți de date. În cazul de față, sistemul proiectat de noi va putea prelua toți cei 16 biți de date deodată, deci ambele intrări $\overline{UB}$ și $\overline{LB}$ vor fi active, adică plasate semnale 0 logic pe ele.

Magistrala de date este bidirecțională. Astfel, la operația de citire după un timp de la generarea comenzilor pentru citire (timpul t_{OE}) magistrala de date va trece din starea HiZ în starea de ieșire, când vor fi disponibile datele memorate la locația accesată.

Pentru operația de scriere formele de undă și tabelul cu valorile sunt prezentate în figura 3.



Parameter	Symbol	K6R4016V1D-08		K6R4016V1D-10		Unit
		Min	Max	Min	Max	
Write Cycle Time	t_{WC}	8	-	10	-	ns
Chip Select to End of Write	t_{CW}	6	-	7	-	ns
Address Set-up Time	t_{AS}	0	-	0	-	ns
Address Valid to End of Write	t_{AW}	6	-	7	-	ns
Write Pulse Width($\overline{OE}$ High)	t_{WP}	6	-	7	-	ns
Write Pulse Width($\overline{OE}$ Low)	t_{WP1}	8	-	10	-	ns
$\overline{UB}$, $\overline{LB}$ Valid to End of Write	t_{BW}	6	-	7	-	ns
Write Recovery Time	t_{WR}	0	-	0	-	ns
Write to Output High-Z	t_{WHZ}	0	4	0	5	ns
Data to Write Time Overlap	t_{DW}	4	-	5	-	ns
Data Hold from Write Time	t_{DH}	0	-	0	-	ns
End of Write to Output Low-Z	t_{OW}	3	-	3	-	ns

Fig.3. Secvența de scriere pentru memoria SRAM Samsung K6R4016V1D (foaie de catalog)

La scrierea memoriei, după stabilirea adresei și datei pe intrare (liniile de date se vor configura ca intrări) se activează pe rând CE și WE în timp ce intrarea OE trebuie să rămână în starea inactivă.

b. Sistemul pentru accesarea memoriei SRAM

Pentru accesul segmentat al memoriei SRAM s-a construit un sistem cu o unitate de execuție pe 16 biți și cu structura prezentată în figura 4.

Sistemul este bazat pe un automat de comandă și control cu memorie ROM, de tipul celui prezentat detaliat în laboratorul anterior. Acesta conține o memorie ROM cu organizarea 256x16, un demultiplexor de funcții 1:16, cu validare sincronă a datelor pe ieșiri, un multiplexor de teste 4:1 și un numărator pe 8 biți cu reîncărcare. Despre structura internă a acestuia s-a tratat detaliat în laboratorul 4 și ea este descrisă în limbajul VHDL în acest laborator. În figura 4 acesta este denumit Procesor.

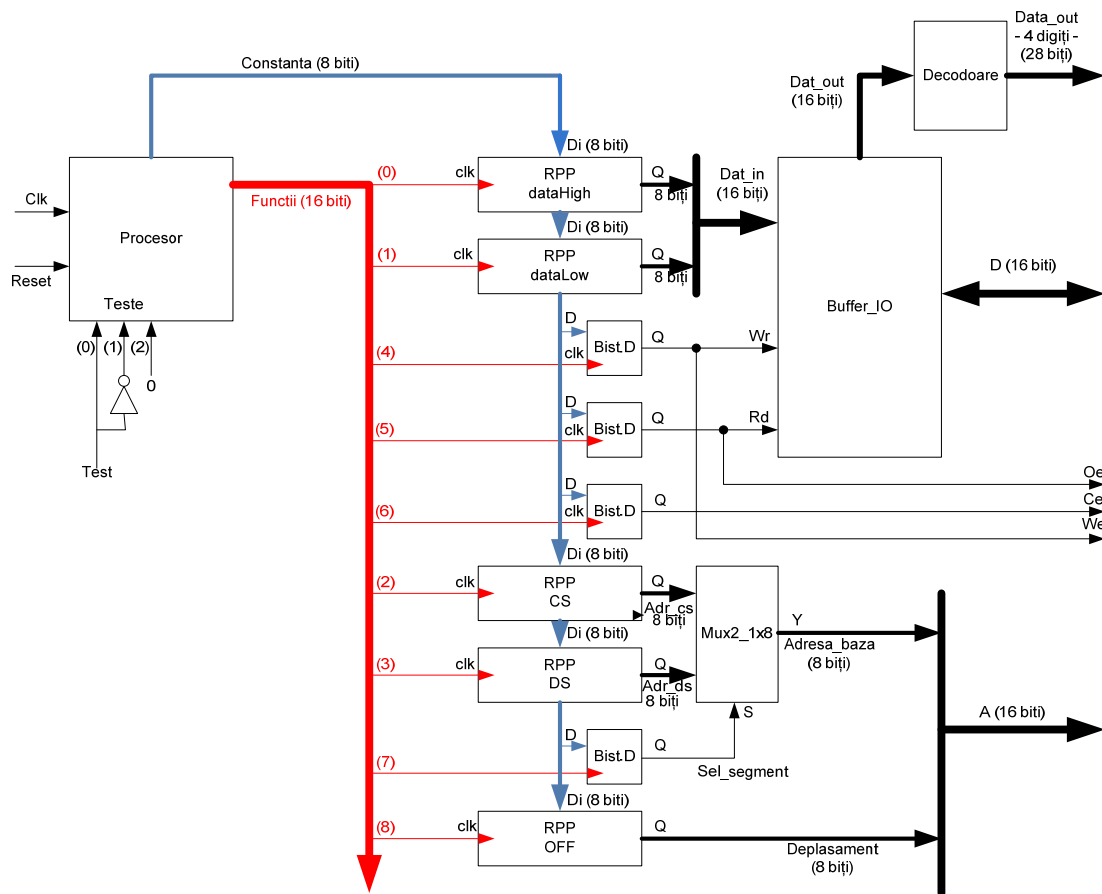


Fig.4. Sistemul pentru adresarea segmentată a memoriei SRAM pe macheta de laborator

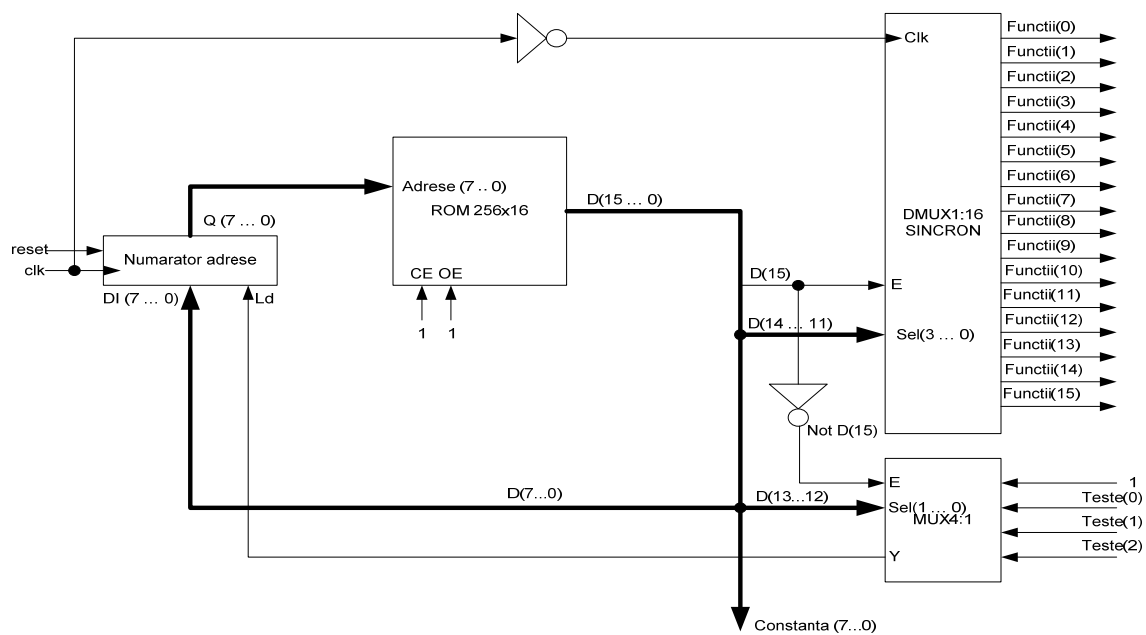


Fig.5. Modulul Procesor

Modulul procesor comandă 5 regiștri cu acces paralel pe 8 biți fiecare. Regiștrii dataHigh și dataLow sunt utilizați pentru stocarea datelor care vor urma să fie scrise în memoria SRAM. Încărcarea acestor regiștri se face prin intermediul unor semnale de ceas generate de procesor. În secțiunea de mai jos se va prezenta locația semnalelor de ceas în structura instrucțiunii. Data pe 8 biți se încarcă prin magistrala Constantă (8 biți) care ajunge ca intrare la regiștrii paraleli de la Procesor.

Regiștrii CS și DS sunt utilizați pentru adresarea segmentată a memoriei. În fiecare se încarcă adresa de bază a segmentului de memorie, utilizând semnale de ceas generate de Procesor și magistrala Constantă. Modul de adresare segmentat al memoriei a fost prezentat în breviarul teoretic al acestui laborator.

Registrul OFF conține deplasamentul în cadrul segmentului de memorie selectat la un moment dat. Încărcarea acestuia se face tot prin comandă venită de la Procesor iar magistrala de date de intrare este conectată la magistrala Constanta.

Alături de regiștrii cu acces paralel sistemul mai conține și 4 bistabili utilizați pentru generarea semnalelor de comandă către memoria SRAM sau pentru selectarea segmentului de memorie care se dorește să se utilizeze la un moment dat.

Sistemul prezentat în figura 4 împarte memoria SRAM în două segmente cu adresele de bază în CS și DS. Deplasarea în cadrul acestora se face prin utilizarea registrului OFF (offset). În acest exemplu, ca un caz particular, adresa de bază se concatenează cu deplasamentul rezultând adresa fizică.

c. Setul de instrucțiuni al sistemului de comandă al memoriei SRAM

Pornind de la schemele din figurile 4 și 5, se poate construi structura instrucțiunii sistemului. Aceasta este prezentată în tabelul de mai jos:

Tabelul 2.

D15	D14	D13	D12	D11	D10	D9	D8	D7	D6	D5	D4	D3	D2	D1	D0
F/ T	selectie functie /				x	x	x		constanta (date) /						
	x	sel. test		x					adresa salt						

Conform figurii 4, fiecare ieșire funcție comandă semnale de ceas către regiștri sau bistabili după cum urmează:

Tabelul 3

Funcție (ieșire DMUX sincron)	Comandă
Funcția 0	Clk dataHigh
Funcția 1	Clk dataLow
Funcția 2	Clk CS
Funcția 3	Clk DS
Funcția 4	Clk bistabil WR
Funcția 5	Clk bistabil RD
Funcția 6	Clk bistabil CE
Funcția 7	Clk bistabil selecție segment
Funcția 8	Clk OFF
Funcția 9 ... Funcția 15	Nealocate

Cele trei intrări de teste sunt conectate după cum urmează:

Tabelul 4:

Intrare test	Test
Test 0	Testează intrarea Test (dacă este 1)
Test 1	Testează intrarea Test (dacă este 0)
Test 2	Permanent 0 (nealocată)

Având funcțiile asociate cu semnalele de ceas ca în tabelul 3 și testele ca în tabelul 4, se definesc următoarele instrucțiuni:

Tabelul 5.

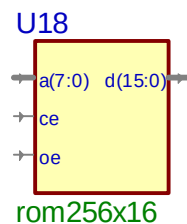
Transfer în regiștri		
Mnemonică instrucțiune	Cod binar și hexa	Descriere
LD DATAH, n8	1000 0000(2) = 80h n8	Încarcă în registrul dataHigh valoarea n8
LD DATAL, n8	1000 1000(2) = 88h n8	Încarcă în registrul dataLow valoarea n8
LD CS, n8	1001 0000(2) = 90h n8	Încarcă în registrul CS valoarea n8
LD DS, n8	1001 1000(2) = 98h n8	Încarcă în registrul DS valoarea n8
LD OFF, n8	1100 0000(2) = C0h n8	Încarcă în registrul OFFset valoarea n8
Setări la nivel de bit		
Mnemonică instrucțiune	Cod binar și hexa	Descriere
BIT WR, b	1010 0000(2) = A0h 0000 000b(2)	Setează starea semnalului WR cu b
BIT RD, b	1010 1000(2) = A8h 0000 000b(2)	Setează starea semnalului RD cu b
BIT CE, b	1011 0000(2) = B0h 0000 000b(2)	Setează starea semnalului CE cu b
BIT SS, b	1011 1000(2) = B8h 0000 000b(2)	Selectează segmentul de memorie: CS dacă b=0 și DS dacă b=1
Teste și salturi		
Mnemonică instrucțiune	Cod binar și hexa	Descriere
JP TEST, adr	0001 0000(2) = 10h adr	Salt dacă intrarea TEST este 1 la adresa adr
JP NTEST, adr	0010 0000(2) = 20h adr	Salt dacă intrarea TEST este 0 la adresa adr
JP adr	0000 0000(2) = 00h adr	Salt necondiționat la adresa adr

Cerințe laborator:

1. Să se descrie în VHDL sistemul de adresare segmentată a memoriei SRAM pentru macheta de laborator.

Rezolvare:

1. Se va crea un proiect nou în Active HDL;
2. Se va crea un fișier rom256x16 unde se va descrie memoria ROM componentă a modului Procesor (figura 5). Structura memoriei ROM este aceeași ca cea descrisă în laboratoarele anterioare cu o modificare doar la dimensiunea datelor (acum pe 16 biți). În interiorul ei se va scrie secvența care va trebui să se execute în continuare!



```
library IEEE;
use IEEE.STD_LOGIC_1164.all;

entity rom256x16 is
    port(
        ce : in STD_LOGIC;
        oe : in STD_LOGIC;
        a : in STD_LOGIC_VECTOR(7 downto 0);
        d : out STD_LOGIC_VECTOR(15 downto 0)
    );
end rom256x16;

architecture rom256x16_a of rom256x16 is
    signal d_int : std_logic_vector(15 downto 0);
    signal enable : std_logic;

begin

    with a select
        d_int <=
            -----aici se scrie programul

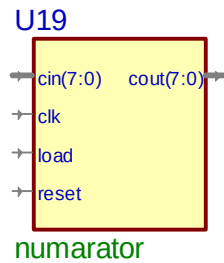
            -----sfarsit program
            x"ffff" when others;

    enable <= ce and oe;

    with enable select
        d <= d_int when '1',
            "ZZZZZZZZZZZZZZZZZZ" when others;

end rom256x16_a;
```

3. Se va crea un fișier numarator.vhd în care se va descrie numărătorul de adrese, componentă a modului Procesor (figura 5).



```
library IEEE;
use IEEE.STD_LOGIC_1164.all;
use IEEE.STD_LOGIC_UNSIGNED.all;

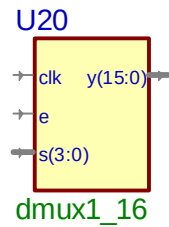
entity numarator is
Port(
    clk : in std_logic;
    reset : in std_logic;
    load : in std_logic;
    cin : in std_logic_vector(7 downto 0);
    cout : out std_logic_vector(7 downto 0)
);
end numarator;

architecture numarator_a of numarator is
begin

process (clk,reset,load)
variable decnt : std_logic_vector(7 downto 0);
begin
    if reset = '0' then
        decnt := "00000000";
    elsif clk'event and clk = '1' then
        if load = '0' then
            decnt := decnt + '1';
        else
            decnt := cin;
        end if;
    end if;
    cout <= decnt;
end process;

end numarator_a;
```

4. Se va crea un fișier dmux1_16.vhd în care se va descrie demultiplexorul sincron. Acesta conține un demultiplexor de tipul 1:16 descris în laboratoarele anterioare și un registru pe ieșire. Ieșirile se vor schimba în ritmul unui semnal de ceas.



```
library IEEE;
use IEEE.STD_LOGIC_1164.all;
```

```
entity dmux1_16 is
    port(
        e : in STD_LOGIC;
        clk : in std_logic;
        s : in STD_LOGIC_VECTOR(3 downto 0);
        y : out STD_LOGIC_VECTOR(15 downto 0)
    );
end dmux1_16;
```

```
architecture dmux1_16_a of dmux1_16 is
    signal y_int : std_logic_vector(15 downto 0);
begin
```

```
    process(e,s)
    begin
        if e = '1' then
            case s is
                when "0000" => y_int <= "0000000000000001";
                when "0001" => y_int <= "0000000000000010";
                when "0010" => y_int <= "0000000000000100";
                when "0011" => y_int <= "0000000000001000";
                when "0100" => y_int <= "0000000000010000";
                when "0101" => y_int <= "0000000000100000";
                when "0110" => y_int <= "0000000001000000";
                when "0111" => y_int <= "0000000010000000";
                when "1000" => y_int <= "0000000100000000";
                when "1001" => y_int <= "0000001000000000";
                when "1010" => y_int <= "0000010000000000";
                when "1011" => y_int <= "0000100000000000";
                when "1100" => y_int <= "0001000000000000";
                when "1101" => y_int <= "0010000000000000";
                when "1110" => y_int <= "0100000000000000";
                when "1111" => y_int <= "1000000000000000";

                when others => y_int <= "0000000000000000";
            end case;
        else
            y_int <= "0000000000000000";
        end if;
    end process;

    process(clk,y_int)
```

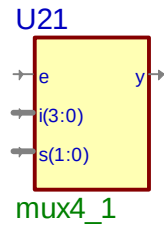
```

begin
    if clk'event and clk='0' then
        y <= y_int;
    end if;
end process;

end dmux1_16_a;

```

5. Se va crea un fișier mux4_1.vhd în care se va descrie multiplexorul de teste. Acesta este un multiplexor 4:1 cu activare descris și în laboratoarele anterioare.



```

library IEEE;
use IEEE.STD_LOGIC_1164.all;

entity mux4_1 is
    port(
        e : in STD_LOGIC;
        i : in STD_LOGIC_VECTOR(3 downto 0);
        s : in STD_LOGIC_VECTOR(1 downto 0);
        y : out STD_LOGIC
    );
end mux4_1;

architecture mux4_1_a of mux4_1 is
begin

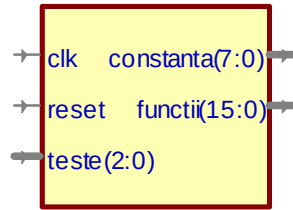
    process(e,i,s)
    begin
        if e = '1' then
            case s is
                when "00" => y <= i(0);
                when "01" => y <= i(1);
                when "10" => y <= i(2);
                when "11" => y <= i(3);
                when others => y <= '0';
            end case;
        else
            y <= '0';
        end if;
    end process;

end mux4_1_a;

```

6. Se va crea un fișier procesor.vhd unde se va descrie procesorul. Acesta este compus din elementele descrise până la acest punct, conectate conform schemei din figura 5. Conexiunile s-au realizat în vhdl utilizând lucrul cu componente.

U1



procesor

```
library IEEE;
use IEEE.STD_LOGIC_1164.all;
```

```
entity procesor is
    port(
        gclk : in STD_LOGIC;
        reset : in STD_LOGIC;
        teste : in STD_LOGIC_VECTOR(2 downto 0);
        constanta : out std_logic_vector(7 downto 0);
        functii : out STD_LOGIC_VECTOR(15 downto 0)
    );
end procesor;
```

```
architecture procesor_a of procesor is
    --declaratii componente
    component rom256x16 is
        port(
            ce : in STD_LOGIC;
            oe : in STD_LOGIC;
            a : in STD_LOGIC_VECTOR(7 downto 0);
            d : out STD_LOGIC_VECTOR(15 downto 0)
        );
    end component;
    component numarator is
        Port(
            clk : in std_logic;
            reset : in std_logic;
            load : in std_logic;
            cin : in std_logic_vector(7 downto 0);
            cout : out std_logic_vector(7 downto 0)
        );
    end component;
    component dmux1_16 is
        port(
            e : in STD_LOGIC;
            clk : in std_logic;
            s : in STD_LOGIC_VECTOR(3 downto 0);
            y : out STD_LOGIC_VECTOR(15 downto 0)
        );
    end component;
    component mux4_1 is
        port(
```

```

        e : in STD_LOGIC;
        i : in STD_LOGIC_VECTOR(3 downto 0);
        s : in STD_LOGIC_VECTOR(1 downto 0);
        y : out STD_LOGIC
    );
end component;
component divizor is
    port(
        clk_in : in STD_LOGIC;
        clk_out : out STD_LOGIC
    );
end component;
--decalratii semnale
signal adr : std_logic_vector(7 downto 0);
signal instr : std_logic_vector(15 downto 0);
signal ld : std_logic;
signal testei : std_logic_vector(3 downto 0);
signal notclk,notinstr : std_logic;
signal clk : std_logic;
begin

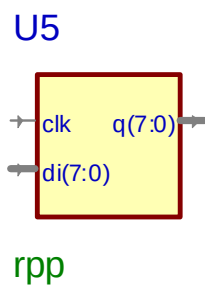
    --memoria rom
    crom : rom256x16 port map('1','1',adr,instr);
    --numaratorul de adrese
    cnumar : numarator port map(clk,reset,ld,instr(7 downto 0),adr);
    --demultiplexorul sincron de functii
    cdmux : dmux1_16 port map(instr(15),notclk,instr(14 downto 11),functii);
    --multiplexorul de teste
    cmux : mux4_1 port map(notinstr,testei,instr(13 downto 12),ld);
    --conexiuni semnale de teste
    testei(0) <= '1';
    testei(3 downto 1) <= testei(2 downto 0);
    --inversare
    notclk <= not clk;
    notinstr <= not instr(15);
    constanta <= instr(7 downto 0);

    cdiv : divizor port map(gclk,clk);

end procesor_a;

```

7. Se va crea un fișier rpp.vhd. Aici se va descrie registrul cu acces paralel pe 8 biți care este multiplicat apoi în schema sistemului, conform figurii 4.



```

library IEEE;
use IEEE.STD_LOGIC_1164.all;

entity rpp is
    port(
        clk : in STD_LOGIC;
        di : in STD_LOGIC_VECTOR(7 downto 0);
        q : out STD_LOGIC_VECTOR(7 downto 0)
    );
end rpp;

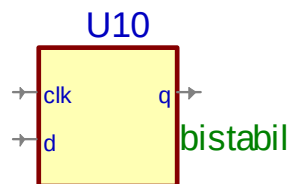
architecture rpp_a of rpp is
begin

    process(clk,di)
    begin
        if clk'event and clk='1' then
            q <= di;
        end if;
    end process;

end rpp_a;

```

8. Se va crea fișierul bistabil.vhd. Aici se va descrie bistabilul D, multiplicat apoi în schema sistemului (figura 4).



```

library IEEE;
use IEEE.STD_LOGIC_1164.all;

entity bistabil is
    port(
        clk : in STD_LOGIC;
        d : in STD_LOGIC;
        q : out STD_LOGIC
    );
end bistabil;

architecture bistabil_a of bistabil is
begin

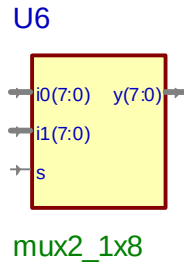
    process(clk,d)
    begin
        if clk'event and clk='1' then
            q <= d;
        end if;
    end process;

end bistabil_a;

```

end bistabil_a;

9. Se va crea fișierul mux2_1x8.vhd în care se va descrie circuitul de selecție al segmentului de memorie (figura 4). Acesta este un multiplexor 2:1 pentru intrări pe 8 biți.



```
library IEEE;
use IEEE.STD_LOGIC_1164.all;

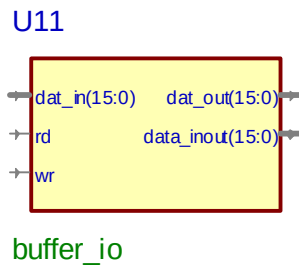
entity mux2_1x8 is
    port(
        s : in STD_LOGIC;
        i0 : in STD_LOGIC_VECTOR(7 downto 0);
        i1 : in STD_LOGIC_VECTOR(7 downto 0);
        y : out STD_LOGIC_VECTOR(7 downto 0)
    );
end mux2_1x8;

architecture mux2_1x8_a of mux2_1x8 is
begin

    with s select
        y <= i1 when '1',
        i0 when others;

end mux2_1x8_a;
```

10. Se va crea fișierul buffer_io.vhd. Acesta va conține bufferul care realizează conversia din magistrala bidirecțională de date de la memoria SRAM în două magistrale unidirecționale: una de intrare și una de ieșire. Magistrala de ieșire va fi conectată la afișoarele 7 segmente în timp ce magistrala de intrare va proveni de la regiștrii de date.



```

library IEEE;
use IEEE.STD_LOGIC_1164.all;

entity buffer_io is
    port(
        rd : in STD_LOGIC;
        wr : in STD_LOGIC;
        dat_in : in STD_LOGIC_VECTOR(15 downto 0);
        dat_out : out STD_LOGIC_VECTOR(15 downto 0);
        data_inout : inout STD_LOGIC_VECTOR(15 downto 0)
    );
end buffer_io;

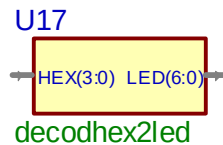
architecture buffer_io_a of buffer_io is
begin

    dat_out<=data_inout when rd='0' and wr='1' else "0000000000000000";
    --"ZZZZZZZZZZZZZZZZZZ";--
    data_inout<=dat_in when rd='1' and wr='0' else "ZZZZZZZZZZZZZZZZZZ";

end buffer_io_a;

```

11. Se va crea fişierul decodor.vhd. Aici se va descrie decodorul binar – 7 segmente necesar pentru afişare.



```

entity decodhex2led is
    port(
        HEX: in      STD_LOGIC_VECTOR (3 downto 0);
        LED: out     STD_LOGIC_VECTOR (6 downto 0)
    );
end decodhex2led;

```

```

architecture decodhex2led_a of decodhex2led is
begin
    with HEX select
    LED<= "0000110" when "0001", --1
        "1011011" when "0010", --2
        "1001111" when "0011", --3
        "1100110" when "0100", --4
        "1101101" when "0101", --5
        "1111101" when "0110", --6
        "0000111" when "0111", --7
        "1111111" when "1000", --8
        "1101111" when "1001", --9
        "1110111" when "1010", --A
        "1111100" when "1011", --b
        "0111001" when "1100", --C
        "1011110" when "1101", --d
        "1111001" when "1110", --E

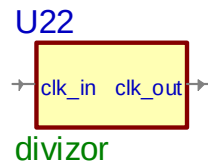
```

```

        "1110001" when "1111", --F
        "0111111" when others; --0
end decodhex2led_a;

```

12. Se va crea fișierul divizor.vhd. Acesta conține divizorul de frecvență pentru semnalul de ceas global.



```

library IEEE;
use IEEE.STD_LOGIC_1164.all;

entity divizor is
    port(
        clk_in : in STD_LOGIC;
        clk_out : out STD_LOGIC
    );
end divizor;

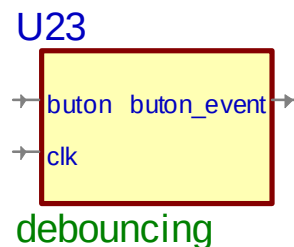
architecture divizor_a of divizor is
begin

    process(clk_in)
        variable cnt_int : integer;
    begin
        if clk_in'event and clk_in='1' then
            cnt_int := cnt_int + 1;
            if cnt_int = 4 then
                clk_out <= '1';
                cnt_int := 0;
            else
                clk_out <= '0';
            end if;
        end if;
    end process;

end divizor_a;

```

13. Se va crea fișierul debouncing.vhd. Acesta va conține circuitul necesar pentru evitarea debouncing-ului (oscilații care apar la acționarea unei taste).



```

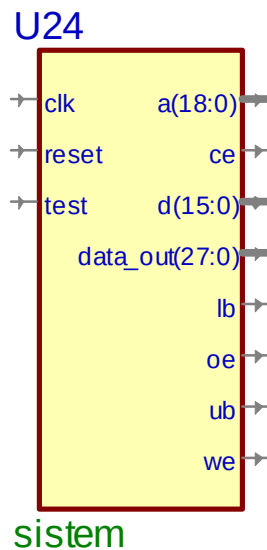
entity debouncing is
    port(
        clk : in STD_LOGIC;
        buton : in STD_LOGIC;
        buton_event : out STD_LOGIC
    );
end debouncing;

architecture debouncing_a of debouncing is
begin

    process(clk,buton)
        variable cnt_int : integer;
    begin
        if clk'event and clk='1' then
            if buton = '1' then
                cnt_int := 0;
                buton_event <= '1';
            else
                if cnt_int = 500000 then
                    buton_event <= '0';
                else
                    buton_event <= '1';
                    cnt_int := cnt_int + 1;
                end if;
            end if;
        end if;
    end process;
end debouncing_a;

```

14. Se va descrie sistemul. Acesta este realizat în VHDL pornind de la figura 4.



```

library IEEE;
use IEEE.STD_LOGIC_1164.all;

entity sistem is
    port(
        clk : in STD_LOGIC;
        reset : in STD_LOGIC;
        test : in std_logic;
        data_out : out std_logic_vector(27 downto 0);
        ce : out STD_LOGIC;
        oe : out STD_LOGIC;

        we : out STD_LOGIC;
        ub : out STD_LOGIC;
        lb : out STD_LOGIC;
        a : out STD_LOGIC_VECTOR(18 downto 0);
        d : inout STD_LOGIC_VECTOR(15 downto 0)
    );
end sistem;

architecture sistem_a of sistem is
    --declaratii componente
    component procesor is
        port(
            gclk : in STD_LOGIC;
            reset : in STD_LOGIC;
            teste : in STD_LOGIC_VECTOR(2 downto 0);
            constanta : out std_logic_vector(7 downto 0);
            functii : out STD_LOGIC_VECTOR(15 downto 0)
        );
    end component;
    component rpp is
        port(
            clk : in STD_LOGIC;
            di : in STD_LOGIC_VECTOR(7 downto 0);
            q : out STD_LOGIC_VECTOR(7 downto 0)
        );
    end component;
    component bistabil is
        port(
            clk : in STD_LOGIC;
            d : in STD_LOGIC;
            q : out STD_LOGIC
        );
    end component;
    component mux2_1x8 is
        port(
            s : in STD_LOGIC;
            i0 : in STD_LOGIC_VECTOR(7 downto 0);
            i1 : in STD_LOGIC_VECTOR(7 downto 0);
            y : out STD_LOGIC_VECTOR(7 downto 0)
        );
    end component;
    component buffer_io is
        port(
            rd : in STD_LOGIC;

```

```

        wr : in STD_LOGIC;
        dat_in : in STD_LOGIC_VECTOR(15 downto 0);
        dat_out : out STD_LOGIC_VECTOR(15 downto 0);
        data_inout : inout STD_LOGIC_VECTOR(15 downto 0)
    );
end component;

```

component decodhex2led is

```

    port(
        HEX: in STD_LOGIC_VECTOR (3 downto 0);
        LED: out STD_LOGIC_VECTOR (6 downto 0)
    );
end component;

```

component debouncing is

```

    port(
        clk : in STD_LOGIC;
        buton : in STD_LOGIC;
        buton_event : out STD_LOGIC
    );
end component;

```

--declaratii semnale

```

signal constanta : std_logic_vector(7 downto 0);
signal functii : std_logic_vector(15 downto 0);
signal adr_cs : std_logic_vector(7 downto 0);
signal adr_ds : std_logic_vector(7 downto 0);
signal adr_off : std_logic_vector(7 downto 0);
signal adresa_baza : std_logic_vector(7 downto 0);
signal wr,rđ : std_logic;
signal dat_in, dat_out : std_logic_vector(15 downto 0);
signal sel_segment : std_logic;
--signal debug : std_logic_vector(15 downto 0);
signal buton_event : std_logic;

```

--conectate la masa

```

signal teste : std_logic_vector(2 downto 0);

```

begin

```

    --debouncing pentru butonul test
    cdeb : debouncing port map(clk,test,buton_event);

```

--procesorul

```

    teste(2) <= '0';--coenctate la masa
    teste(1) <= not buton_event;
    teste(0) <= buton_event;
    cprocesor : procesor port map(clk,reset,teste,constanta,functii);
    --debug(15 downto 8) <= "00000000";
    --debug(7 downto 0) <= dat_out(7 downto 0);
    --debug(15 downto 8) <= adresa_baza;

```

--registrul dataHigh

```

    dataHigh : rpp port map(functii(0),constanta,dat_in(15 downto 8));

```

--registrul dataLow

```

    dataLow : rpp port map(functii(1),constanta,dat_in(7 downto 0));

```

--registrul Code Segment

```

CS : rpp port map(funcții(2),constanta,adr_cs);
--registrul Data Segment
DS : rpp port map(funcții(3),constanta,adr_ds);
--registru offset
OFF : rpp port map(funcții(8),constanta,adr_off);

--bistabilul pentru semnalul wr
bwr : bistabil port map(funcții(4),constanta(0),wr);
we <= wr; --conectare port we la semnalul wr
--bistabilul pentru semnalul rd
brd : bistabil port map(funcții(5),constanta(0),rd);
oe <= rd; --conectare port oe la semnalul rd
--bistabilul pentru portul ce
bce : bistabil port map(funcții(6),constanta(0),ce);
--bistabilul pentru selecția segmentului
bseg : bistabil port map(funcții(7), constanta(0),sel_segment);

--circuitul de selecție segment
cmux2_1 : mux2_1x8 port map(sel_segment,adr_cs,adr_ds,adresa_baza);

--buffer-ul tree state pentru date
cbuffio : buffer_io port map(rd,wr,dat_in,dat_out,d);

--decodările 7 segmente pentru afișarea ieșirii
decodare: for i in 0 to 3 generate
    dcifra : decodhex2led port map(dat_out(4*i+3 downto 4*i),data_out(7*i+6 downto
7*i));--(debug(4*i+3 downto 4*i),data_out(7*i+6 downto 7*i));--
end generate;

--adresa memorie
a(15 downto 8) <= adresa_baza;
a(7 downto 0) <= adr_off;

--alte semnale conectate la masă
ub <= '0';
lb <= '0';
a(18 downto 16) <= "000";

end sistem_a;

```

2. Să se implementeze în FPGA pe machetele din laborator sistemul descris la punctul 1 astfel încât acesta să adreseze segmentat memoria SRAM de pe machetă și în plus, pe baza programului conținut în memoria ROM, să efectueze următoarele operații:

Organizează memoria SRAM cu segmentul CS la adresa de bază 00h și segmentul DS la adresa de bază 80h.

Scrie în memoria SRAM următoarele informații:

Segmentul CS, adresa 00h, data 0055h.

Segmentul CS adresa 01h, data 00AAh.

Citește din memoria SRAM pe rând informațiile de la adresa 00h segmentul CS și adresa 01h segmentul CS.

Rezolvare

Cerințele de la acest punct se rezolva prin programarea memoriei ROM cu programul care să execute operațiile sus menționate.

Pe baza setului de instrucțiuni definit în tabelul 5 se scrie în memoria ROM (fișierul rom256x16.vhd, în secțiunea încadrată de comentariile „aici se scrie programul” și „sfârșit program”) programul.

-----aici se scrie programul

```
x"B001" when x"00",--BIT CE,1
x"A001" when x"01",--BIT WR,1
x"A801" when x"02",--BIT RD,1
x"9880" when x"03",--LD DS,80H
x"9000" when x"04",--LD CS,00H
x"8000" when x"05",--LD DATAH,00H
x"8855" when x"06",--LD DATAL,55H
x"C000" when x"07",--LD OFF,00H
x"B800" when x"08",--BIT SS,0
x"B000" when x"09",--BIT CE,0
x"A000" when x"0a",--BIT WR,0
x"100b" when x"0b",--JP TEST,$
x"200c" when x"0c",--JP NTEST,$
x"A001" when x"0d",--BIT WR,1
x"B001" when x"0e",--BIT CE,1
x"8000" when x"0f",--LD DATAH,00H
x"88AA" when x"10",--LD DATAL,AAH
x"C001" when x"11",--LD OFF,01H
x"B000" when x"12",--BIT CE,0
x"A000" when x"13",--BIT WR,0
x"1014" when x"14",--JP TEST,$
x"2015" when x"15",--JP NTEST,$
x"A001" when x"16",--BIT WR,1
x"B001" when x"17",--BIT CE,1
x"C000" when x"18",--LD OFF,00H
x"B000" when x"19",--BIT CE,0
x"A800" when x"1a",--BIT RD,0
x"101b" when x"1b",--JP TEST,$
x"201c" when x"1c",--JP NTEST,$
x"A801" when x"1d",--BIT RD,1
x"B001" when x"1e",--BIT CE,1
x"C001" when x"1f",--LD OFF,01H
x"B000" when x"20",--BIT CE,0
x"A800" when x"21",--BIT RD,0
x"1022" when x"22",--JP TEST,$
```

```
x"2023" when x"23",--JP NTEST,$
x"A801" when x"24",--BIT RD,1
x"B001" when x"25",--BIT CE,1
x"0026" when x"26",--JP TEST,$
-----sfarsit program
```

În primul rând, se inițializează semnalele WE, RD și CE în 1 logic, adică în starea lor inactivă (întrucât la pornirea FPGA-ului toate ieșirile sale sunt puse în 0 logic).

```
x"B001" when x"00",--BIT CE,1
x"A001" when x"01",--BIT WR,1
x"A801" when x"02",--BIT RD,1
```

Pasul următor sunt inițializați regiștrii DS și CS în conformitate cu cerința problemei. Segmentul CS are adresa de bază 00h deci registrul CS este încărcat cu valoarea 00h. Pe de altă parte, segmentul DS are adresa de bază 80h deci registrul segment DS trebuie încărcat cu 80h.

```
x"9880" when x"03",--LD DS,80H
x"9000" when x"04",--LD CS,00H
```

În continuare trebuie pregătită data care se dorește scrisă prima dată:

```
x"8000" when x"05",--LD DATAH,00H
x"8855" when x"06",--LD DATAL,55H
```

Deplasamentul este setat la 00h:

```
x"C000" when x"07",--LD OFF,00H
```

Și se selectează segmentul CS:

```
x"B800" when x"08",--BIT SS,0
```

După finalizarea setării adresei, selectării segmentului și setării datelor sunt activate semnalele de comandă pentru memoria SRAM și anume activarea circuitului și activarea scrierii:

```
x"B000" when x"09",--BIT CE,0
x"A000" when x"0a",--BIT WR,0
```

După activarea scrierii se lasă o temporizare în care semnalul WR să rămână activ. În cazul nostru, pentru temporizare se va utiliza intrarea de test care va proveni de la un buton de pe machetă. Prin acesta, se așteaptă apăsarea și ridicarea butonului de test. Această așteptare este echivalent în acest caz cu o temporizare.

```
x"100b" when x"0b",--JP TEST,$  
x"200c" when x"0c",--JP NTEST,$
```

După temporizare, sunt dezactivate semnalele de comandă scriere:

```
x"A001" when x"0d",--BIT WR,1  
x"B001" when x"0e",--BIT CE,1
```

Similar se procedează la scrierea următoarei adrese (deplasament 01h) cu data 00AAh în același segment CS:

```
x"8000" when x"0f",--LD DATAH,00H  
x"88AA" when x"10",--LD DATAL,AAH  
x"C001" when x"11",--LD OFF,01H  
x"B000" when x"12",--BIT CE,0  
x"A000" when x"13",--BIT WR,0  
x"1014" when x"14",--JP TEST,$  
x"2015" when x"15",--JP NTEST,$  
x"A001" when x"16",--BIT WR,1  
x"B001" when x"17",--BIT CE,1
```

Pentru citire se introduce adresa unde se dorește să se efectueze citirea în cadrul aceluiasi segment CS:

```
x"C000" when x"18",--LD OFF,00H
```

Și sunt activate apoi semnalele de comandă citire pentru memoria SRAM:

```
x"B000" when x"19",--BIT CE,0  
x"A800" when x"1a",--BIT RD,0
```

Pentru a putea vizualiza informația citită (în mod normal trebuie să fie vizibilă pe afișoarele 7 segmente), se va crea o temporizare prin așteptarea apăsării și ridicării butonului de test.

```
x"101b" when x"1b",--JP TEST,$  
x"201c" when x"1c",--JP NTEST,$
```

În final sunt dezactivate comenzile pentru citire:

```
x"A801" when x"1d",--BIT RD,1  
x"B001" when x"1e",--BIT CE,1
```

Similar se citește și următoarea locație de la adresa 01h, segmentul CS:

```
x"C001" when x"1f",--LD OFF,01H
```

```

x"B000" when x"20",--BIT CE,0
x"A800" when x"21",--BIT RD,0
x"1022" when x"22",--JP TEST,$
x"2023" when x"23",--JP NTEST,$
x"A801" when x"24",--BIT RD,1
x"B001" when x"25",--BIT CE,1
x"0026" when x"26",--JP TEST,$

```

Cu programul scris în memoria ROM se va sintetiza și implementa sistemul. La alocarea constrângerilor se vor asocia următorii pini conform descrierii într-un fișier de constrângeri de mai jos:

```

NET "a<0>" LOC = "l6" ;
NET "a<10>" LOC = "u2" ;
NET "a<11>" LOC = "v3" ;
NET "a<12>" LOC = "v1" ;
NET "a<13>" LOC = "w1" ;
NET "a<14>" LOC = "v2" ;
NET "a<15>" LOC = "v5" ;
NET "a<16>" LOC = "v4" ;
NET "a<17>" LOC = "u5" ;
NET "a<18>" LOC = "u6" ;
NET "a<1>" LOC = "k4" ;
NET "a<2>" LOC = "h5" ;
NET "a<3>" LOC = "g6" ;
NET "a<4>" LOC = "f3" ;
NET "a<5>" LOC = "g1" ;
NET "a<6>" LOC = "g2" ;
NET "a<7>" LOC = "k3" ;
NET "a<8>" LOC = "t2" ;
NET "a<9>" LOC = "t1" ;
NET "ce" LOC = "l5" ;
NET "clk" LOC = "AA12" ;
NET "d<0>" LOC = "l4" ;
NET "d<10>" LOC = "m6" ;
NET "d<11>" LOC = "n2" ;
NET "d<12>" LOC = "n1" ;
NET "d<13>" LOC = "m2" ;
NET "d<14>" LOC = "m1" ;
NET "d<15>" LOC = "l2" ;
NET "d<1>" LOC = "l3" ;
NET "d<2>" LOC = "m5" ;
NET "d<3>" LOC = "m4" ;
NET "d<4>" LOC = "m3" ;
NET "d<5>" LOC = "n4" ;
NET "d<6>" LOC = "n3" ;

```

```

NET "d<7>" LOC = "t5" ;
NET "d<8>" LOC = "t4" ;
NET "d<9>" LOC = "t6" ;
NET "data_out<0>" LOC = "b17" ;
NET "data_out<10>" LOC = "d14" ;
NET "data_out<11>" LOC = "d13" ;
NET "data_out<12>" LOC = "b13" ;
NET "data_out<13>" LOC = "e13" ;
NET "data_out<14>" LOC = "b10" ;
NET "data_out<15>" LOC = "a12" ;
NET "data_out<16>" LOC = "c10" ;
NET "data_out<17>" LOC = "a9" ;
NET "data_out<18>" LOC = "b9" ;
NET "data_out<19>" LOC = "a10" ;
NET "data_out<1>" LOC = "b18" ;
NET "data_out<20>" LOC = "e9" ;
NET "data_out<21>" LOC = "c6" ;
NET "data_out<22>" LOC = "b8" ;
NET "data_out<23>" LOC = "e7" ;
NET "data_out<24>" LOC = "c5" ;
NET "data_out<25>" LOC = "e6" ;
NET "data_out<26>" LOC = "b5" ;
NET "data_out<27>" LOC = "a4" ;
NET "data_out<2>" LOC = "a18" ;
NET "data_out<3>" LOC = "b15" ;
NET "data_out<4>" LOC = "d17" ;
NET "data_out<5>" LOC = "c17" ;
NET "data_out<6>" LOC = "e17" ;
NET "data_out<7>" LOC = "e15" ;
NET "data_out<8>" LOC = "e16" ;
NET "data_out<9>" LOC = "a14" ;
NET "lb" LOC = "l1" ;
NET "oe" LOC = "k1" ;
NET "reset" LOC = "Y17" ;
NET "test" LOC = "D1" ;
NET "ub" LOC = "k2" ;
NET "we" LOC = "u4" ;

```

3. Utilizând sistemul și segmentarea de la problema 2 (CS adresa de bază 00h, DS adresă de bază 80h) să se implementeze pe machetă sistemul care să efectueze următoarele operații în memoria SRAM:
Scrie segment DS, adresa 00h, data 0012h
Scrie segment DS, adresa 01h, data 0034h
Citește segment DS, adresa 00h

Citește segment DS, adresa 01h

Rezolvare

Cerințele de la acest punct se rezolvă prin reprogramarea memoriei ROM cu programul care să execute operațiile sus menționate. Acesta este următorul:

-----aici se scrie programul

```
x"B001" when x"00",--BIT CE,1
x"A001" when x"01",--BIT WR,1
x"A801" when x"02",--BIT RD,1
x"9880" when x"03",--LD DS,80H
x"9000" when x"04",--LD CS,00H
x"8000" when x"05",--LD DATAH,00H
x"8812" when x"06",--LD DATAL,12H
x"C000" when x"07",--LD OFF,00H
x"B801" when x"08",--BIT SS,1
x"B000" when x"09",--BIT CE,0
x"A000" when x"0a",--BIT WR,0
x"100b" when x"0b",--JP TEST,$
x"200c" when x"0c",--JP NTEST,$
x"A001" when x"0d",--BIT WR,1
x"B001" when x"0e",--BIT CE,1
x"8000" when x"0f",--LD DATAH,00H
x"8834" when x"10",--LD DATAL,34H
x"C001" when x"11",--LD OFF,01H
x"B000" when x"12",--BIT CE,0
x"A000" when x"13",--BIT WR,0
x"1014" when x"14",--JP TEST,$
x"2015" when x"15",--JP NTEST,$
x"A001" when x"16",--BIT WR,1
x"B001" when x"17",--BIT CE,1
x"C000" when x"18",--LD OFF,00H
x"B000" when x"19",--BIT CE,0
x"A800" when x"1a",--BIT RD,0
x"101b" when x"1b",--JP TEST,$
x"201c" when x"1c",--JP NTEST,$
x"A801" when x"1d",--BIT RD,1
x"B001" when x"1e",--BIT CE,1
x"C001" when x"1f",--LD OFF,01H
x"B000" when x"20",--BIT CE,0
x"A800" when x"21",--BIT RD,0
x"1022" when x"22",--JP TEST,$
x"2023" when x"23",--JP NTEST,$
x"A801" when x"24",--BIT RD,1
x"B001" when x"25",--BIT CE,1
x"0026" when x"26",--JP TEST,$
```

-----sfarsit program

4. Utilizând sistemul și segmentarea de la problema 2 (CS adresa de bază 00h, DS adresă de bază 80h) să se implementeze pe machetă sistemul care să efectueze următoarele operații în memoria SRAM:

Citește segment CS, adresa 01h

Citește segment DS, adresa 00h

Rezolvare

Cerințele de la acest punct se rezolva prin reprogramarea memoriei ROM cu programul care să execute operațiile sus menționate. Acesta este următorul:

```
-----aici se scrie programul
x"B001" when x"00",--BIT CE,1
x"A001" when x"01",--BIT WR,1
x"A801" when x"02",--BIT RD,1
x"9880" when x"03",--LD DS,80H
x"9000" when x"04",--LD CS,00H
x"C001" when x"05",--LD OFF,01H
x"B800" when x"06",--BIT SS,0
x"B000" when x"07",--BIT CE,0
x"A800" when x"08",--BIT RD,0
x"1009" when x"09",--JP TEST,$
x"200a" when x"0a",--JP NTEST,$
x"A801" when x"0b",--BIT RD,1
x"B001" when x"0c",--BIT CE,1
x"C000" when x"0d",--LD OFF,00H
x"B801" when x"0e",--BIT SS,1
x"B000" when x"0f",--BIT CE,0
x"A800" when x"10",--BIT RD,0
x"1011" when x"11",--JP TEST,$
x"2012" when x"12",--JP NTEST,$
x"A801" when x"13",--BIT RD,1
x"B001" when x"14",--BIT CE,1
x"0015" when x"15",--JP TEST,$
-----sfarsit program
```

Temă

5. Să se analizeze codul în VHDL care descrie toate componentele și sistemele din proiectul realizat la punctele 1 și 2 și să se traseze schemele detaliate ale modului Procesor și a sistemului.
6. În condițiile de la punctele 1..4, să se efectueze programele pentru următoarele operații:

Program 1:

Scrie segment CS, adresa 02h, data 0078h
Scrie segment DS, adresa 04h, data 0090h
Citește segment CS, adresa 02h
Citește segment DS, adresa 04h

Program 2:

Scrie segment CS, adresa 20h, data 1234h
Scrie segment DS, adresa 24h, data 5678h
Citește segment DS, adresa 24h
Citește segment CS, adresa 20h

Program 3:

Scrie segment CS, adresa 02h, data 0078h
Scrie segment CS, adresa 04h, data 0090h
Citește segment CS, adresa 02h
Citește segment CS, adresa 04h

Program 4:

Scrie segment DS, adresa 34h, data 2345h
Scrie segment DS, adresa 56h, data 6789h
Citește segment DS, adresa 34h
Citește segment DS, adresa 56h

Program 5:

Scrie segment CS, adresa 44h, data aa55h
Scrie segment DS, adresa 55h, data 2233h
Citește segment CS, adresa 44h
Citește segment DS, adresa 55h

Program 6:

Scrie segment CS, adresa 80h, data 7820h
Scrie segment DS, adresa 80h, data 0090h
Citește segment CS, adresa 80h
Citește segment DS, adresa 80h

Program 7:

Scrie segment CS, adresa 12h, data 0078h
Scrie segment DS, adresa 12h, data 0077h

Citește segment CS, adresa 12h

Citește segment DS, adresa 12h

7. Să se proiecteze un sistem de adresare segmentată a memoriei SRAM 256k x 16 care să o împartă în 4 segmente: CS, DS, SS, ES. Segmentele accesează fiecare începând cu adresele de bază 00h, 40h, 80h, C0h.

Criterii de evaluare:
